

ベクトルと行列

NumPyで線形代数

rkskmt

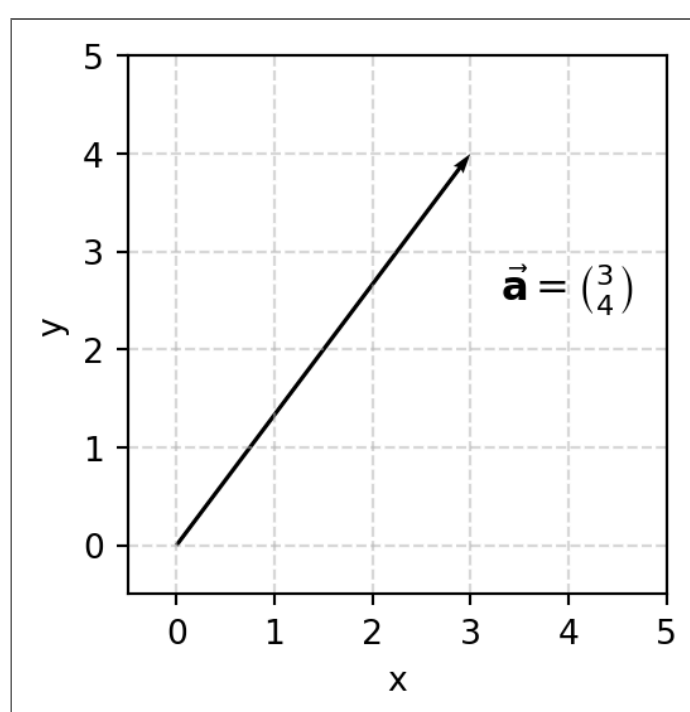
1

ベクトルのおさらい

2

ベクトル（ベクター・Vector）とは

- 「**大きさ**」と「**方向**」の組み合わせ
 - **速度・力** など
 - **位置** は **無関係**
 - **大きさのみ** は **スカラー（scalar）**



3

表記方法

- 表記は成分を縦に並べて書くことが多い
- これを単に小文字太文字の**a**として記号表記する
 - 高校での $\vec{a}$ から矢印を省略した形

高校での表記

$$\vec{a} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$$

大学・論文・技術文章の主流

$$\mathbf{a} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$$

4

NumPyでベクトルを表現・演算

5

ベクトルの表現

- NumPyでは配列（1次元配列）でベクトルを表現
 - 1次元でも **ndarray** の扱い
 - ▶ 1D array
 - 縦ベクトルでないが問題ない

```
1 import numpy as np
2
3 a = np.array([3, 4])
4 print(a) # [3, 4]と横表記だが気にしない
5 print(type(a)) # <class 'numpy.ndarray'>
```

Python

6

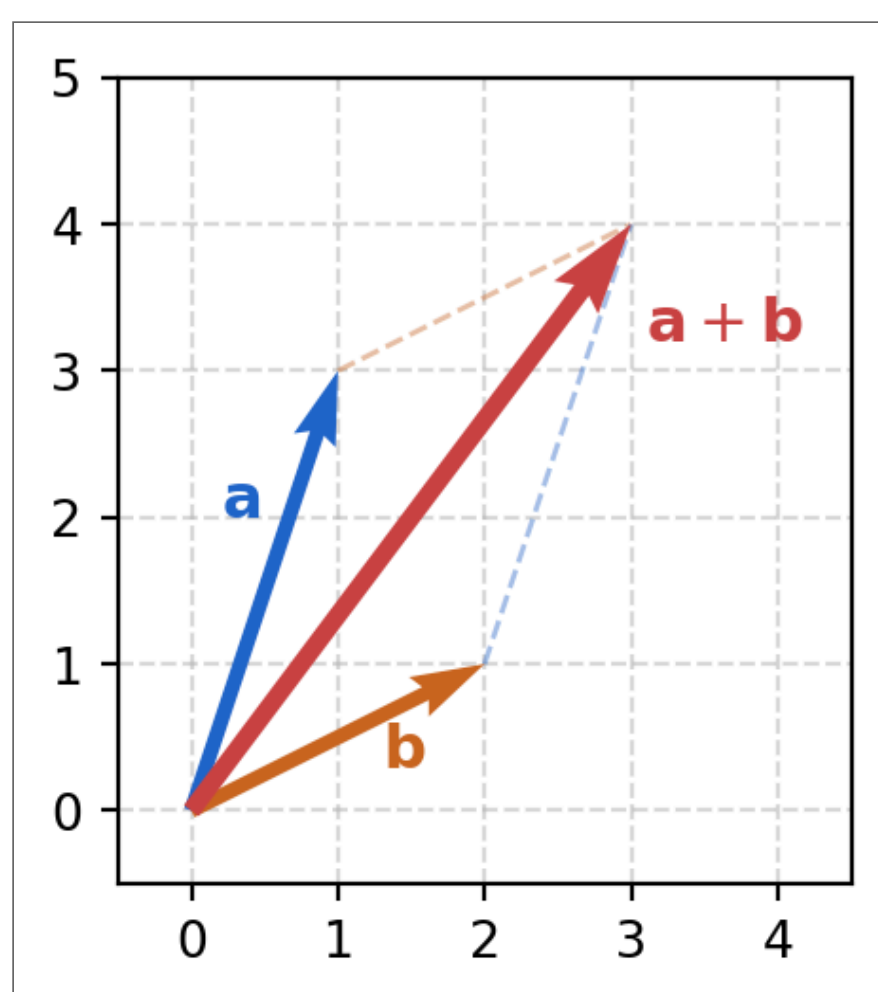
ベクトルの足し算

- 足し算は**成分ごと**に行う
- 入力：「**ベクトル**と**ベクトル**」
- 出力：「**ベクトル**」
 - 引き算も同様

$$\begin{pmatrix} 1 \\ 3 \end{pmatrix} + \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$$

```
1 import numpy as np
2
3 a = np.array([1, 3])
4 b = np.array([2, 1])
5
6 print(a + b) # [3 4]
7 print(a - b) # [-1 2]
```

Python



7

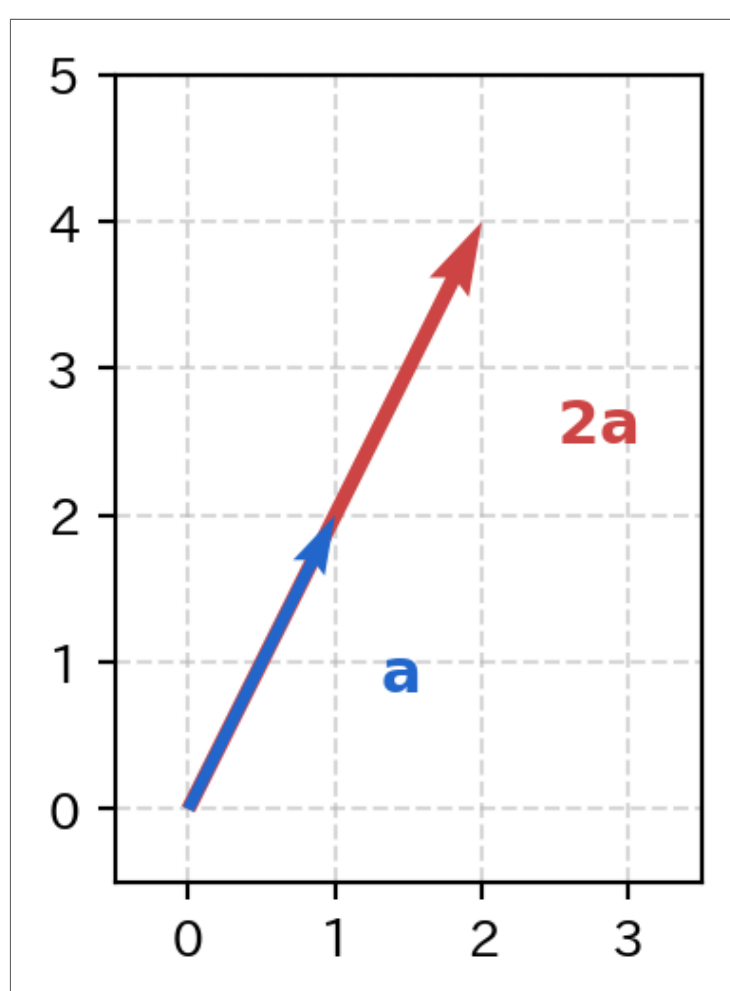
ベクトルの掛け算 1（スカラー倍）

- 掛け算は**全成分それぞれ** に**スカラー値**を掛ける
 - 入力：「**ベクトル**と**スカラー**」
 - 出力：「**ベクトル**」
- 割り算も同じ

$$\begin{pmatrix} 1 \\ 2 \end{pmatrix} \times 2 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}$$

```
1 import numpy as np
2
3 a = np.array([1, 2])
4
5 print(a * 2)    # [2 4]
```

Python



8

ベクトルの掛け算 2（内積）

- **内積**（inner product、dot product）は、対応成分同士をかけて足す
- 入力：「**ベクトル**と**ベクトル**」
- 出力：「**スカラー**」

$$\begin{pmatrix} 1 \\ 2 \end{pmatrix} \cdot \begin{pmatrix} 3 \\ 4 \end{pmatrix} = 1 \times 3 + 2 \times 4 = 11$$

9

【重要】 NumPyにおける内積の演算子

- 2つの **ndarray** を **@** という二項演算子に入力
→ 普通なら ***** とするところを **@** に変える

Python

```
1 import numpy as np
2
3 a = np.array([1, 2])
4 b = np.array([3, 4])
5
6 # 内積
7 print(a @ b) # 11
8
9 # * 演算子だと内積にならない
10 print(a * b) # [1*3, 2*4] → [3 8]
```

⚠ 注意

- *** で演算しても **エラーにはならないので危険**

行列

行列

- ベクトルを横方向に並べたもの

→
$$\vec{a} = \begin{pmatrix} 1 \\ 3 \end{pmatrix}, \quad \vec{b} = \begin{pmatrix} 2 \\ 4 \end{pmatrix} \Rightarrow A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

- 3 × 4 行列の例

→
$$\vec{a} = \begin{pmatrix} 1 \\ 4 \\ 7 \end{pmatrix}, \quad \vec{b} = \begin{pmatrix} 2 \\ 5 \\ 8 \end{pmatrix}, \quad \vec{c} = \begin{pmatrix} 3 \\ 6 \\ 9 \end{pmatrix}, \quad \vec{d} = \begin{pmatrix} 10 \\ 11 \\ 12 \end{pmatrix} \Rightarrow A = \begin{pmatrix} 1 & 2 & 3 & 10 \\ 4 & 5 & 6 & 11 \\ 7 & 8 & 9 & 12 \end{pmatrix}$$

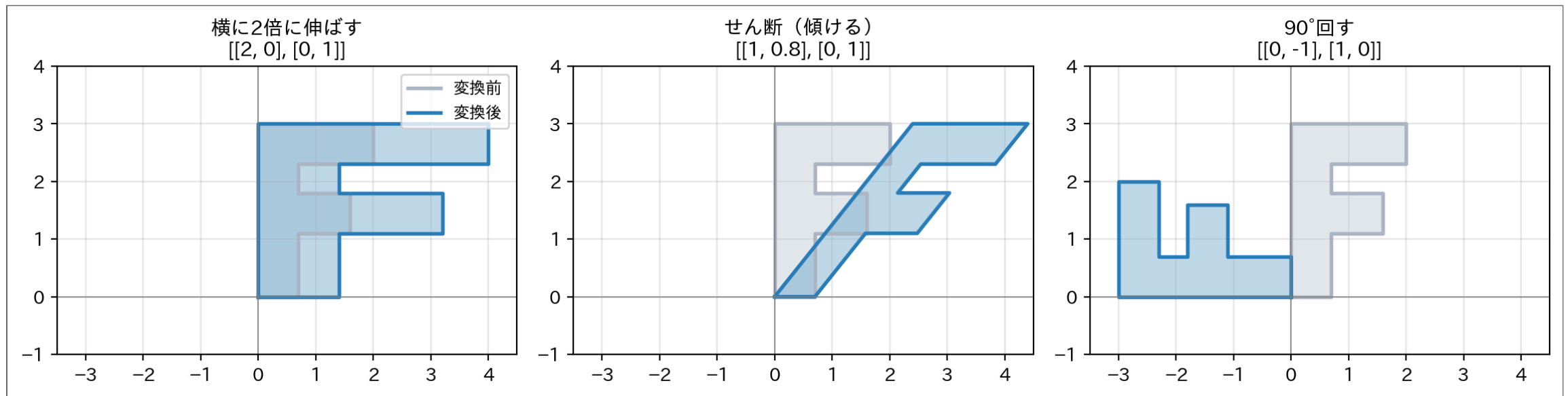
行列とベクトルの積

- 行列の列数とベクトルの次元が同じ場合、掛け算することが可能
→ それぞれの **行** と **ベクトルの内積** を **縦に並べる**

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 \\ 6 \end{pmatrix} = \begin{pmatrix} \quad \\ \quad \end{pmatrix}$$

計算の意味：行列は点を「動かす機械」

- さっきの掛け算、何をしていたのか？ — 行列はベクトル（点）を別の場所へ動かす変換
- 形（F）にかけてみると、その正体が見える

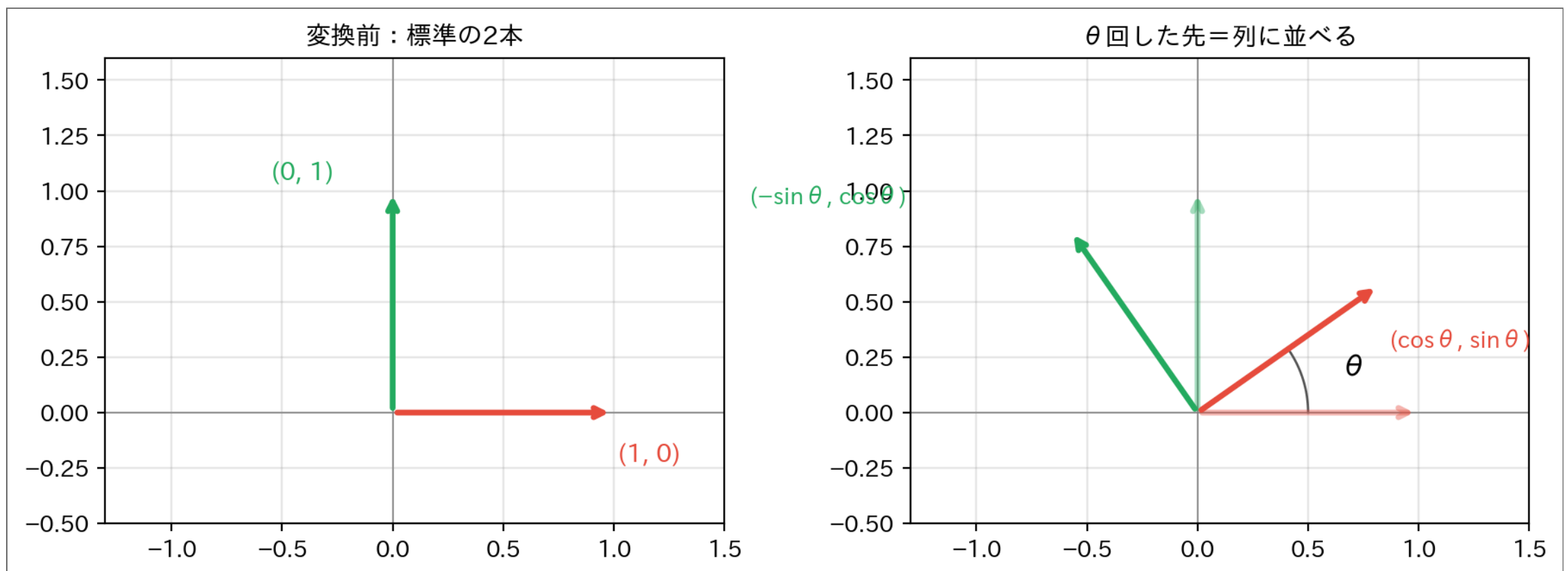


- 同じ「行列×ベクトル」でも、行列を変えれば**伸ばす・傾ける・回す**と動きが変わる
- 行列とは「**空間そのものをどう変形するか**」を表す数の表

14

なぜ回転行列は $\cos$ と $\sin$ なのか

- 行列の**列**には、深い意味がある： $(1, 0)$ と $(0, 1)$ の行き先を並べただけ



- $(1, 0)$ を θ 回すと $(\cos \theta, \sin \theta)$ 、 $(0, 1)$ を回すと $(-\sin \theta, \cos \theta)$
- これを**列に並べる**と、みんな知っている回転行列：

$$R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$

💡 ヒント

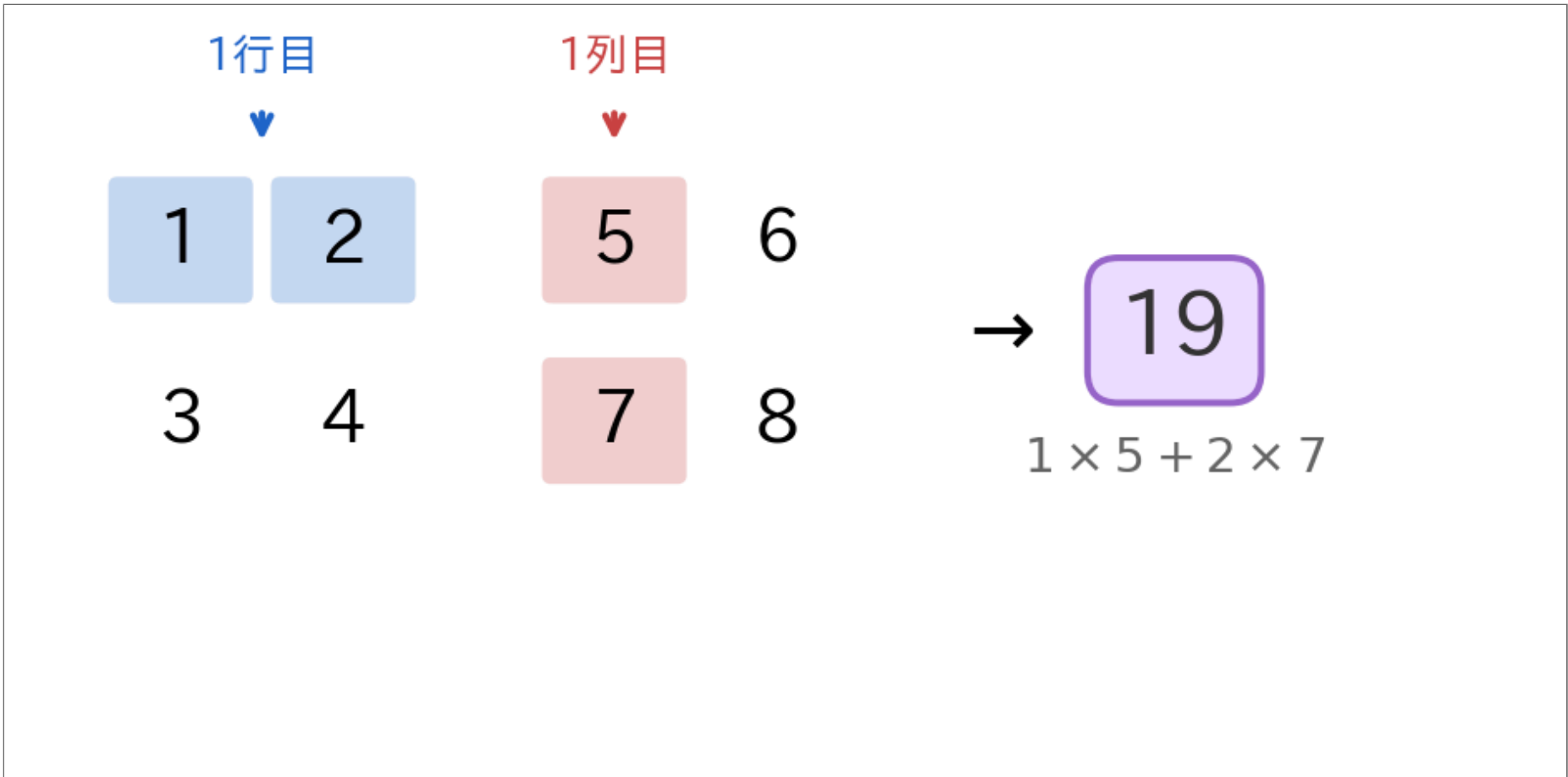
「行列の列＝基底ベクトルの行き先」と知っていれば、**変換を見れば行列を、行列を見れば変換を書ける**。拡大は対角行列 $\begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix}$ 、回転は上の $R(\theta)$ — この2つの組み合わせが変換の基本。

15

行列と行列の積

- 行列同士の掛け算は『**行×列**』の内積を並べる

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 1 \cdot 5 + 2 \cdot 7 & 1 \cdot 6 + 2 \cdot 8 \\ 3 \cdot 5 + 4 \cdot 7 & 3 \cdot 6 + 4 \cdot 8 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$$



16

NumPyで行列を表現・演算

17

NumPyでの行列

- ndarrayで作成可能だが、**行ごとに宣言**する点に注意
 - 縦ベクトルの並びではないので少し気持ち悪い（仕様）
 - **「行」ごとに宣言**して縦にスタックしていく
 - 以下の場合でも **[[1 3][2 4]]**ではなく**[[1 2][3 4]]**

$$\vec{a} = \begin{pmatrix} 1 \\ 3 \end{pmatrix}, \quad \vec{b} = \begin{pmatrix} 2 \\ 4 \end{pmatrix} \Rightarrow A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

```
1 import numpy as np
2
3 A = np.array([[1, 2],
4               [3, 4]])
5
6 # アクセス方法
7 print(A[0, 1]) # 1行目2列目 → 2
8 print(A[1, :]) # 2行目全体 → [3 4]
9 print(A[:, 0]) # 1列目全体 → [1 3]
```

① ノート

- 行優先（**row-major**）ではなく列優先（**column-major**）の実装もありうる（OpenGLなど）
- ただ、現在の言語では行優先が大半

18

NumPyでの行列の演算

- ベクトルの内積と同じく `@` を使用

```
1 import numpy as np
2
3 # A: Matrix b:Vector
4 A = np.array([[1, 2], [3, 4]])
5 b = np.array([5, 6])
6
7 print(A @ b)  # [17 39]
8
9
10 # A:Matrix B:Matrix
11 A = np.array([[1, 2], [3, 4]])
12 B = np.array([[5, 6], [7, 8]])
13
14 print(A @ B)  # [[19 22] [43 50]]
```

Python

19

もし `*` を使用するとどうになってしまうのか

- `@` : 積（行列積）
- `*` : **要素ごと** の掛け算
 - Hadamard積（ハダマール積）
 - 画像処理等である要素をマスクするときに使用される

```
1 import numpy as np
2
3 A = np.array([[1, 2], [3, 4]])
4 B = np.array([[5, 6], [7, 8]])
5
6 print(A * B)
7 # [[ 5 12]
8 #  [21 32]] ← 同じ位置同士をかけただけ
9
10 print(A @ B)
11 # [[19 22]
12 #  [43 50]] ← 行列積
```

Python

20

逆行列

- **逆行列**：行列 A に対して $AA^{-1} = I$ （単位行列）を満たす A^{-1}
 - 元の数と掛けたら 1 になる **逆数** のようなもの
- NumPyでは **np.linalg.inv()** という便利関数で逆行列が求まる
 - **linalg** = **linear algebra**（線形代数）の略

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \Rightarrow A^{-1} = \begin{pmatrix} -2 & 1 \\ 1.5 & -0.5 \end{pmatrix}$$

```
1 import numpy as np
2
3 A = np.array([[1, 2], [3, 4]])
4 A_inv = np.linalg.inv(A)
5 print(A_inv) # [[-2.   1. ] [ 1.5 -0.5]]
6 print(A @ A_inv) # [[1. 0.] [0. 1.]] ← 単位行列
```

Python

21

@ 演算子で方程式を解こう

22

行列で連立方程式を表現

以下の連立方程式は

$$\begin{cases} x - y = 1 \\ x + y = 3 \end{cases}$$

行列で以下のように表現可能

$$\underbrace{\begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}}_A \underbrace{\begin{pmatrix} x \\ y \end{pmatrix}}_{\mathbf{x}} = \underbrace{\begin{pmatrix} 1 \\ 3 \end{pmatrix}}_{\mathbf{b}}$$

- これは $A\mathbf{x} = \mathbf{b}$ の形である

23

NumPyで $Ax = b$ の連立方程式を解く

- $Ax = b$ の両辺に左から A^{-1} を掛ける

$$Ax = b$$
$$A^{-1}Ax = A^{-1}b$$
$$x = A^{-1}b$$

- $A^{-1}b$ が解

```
1 import numpy as np
2
3 A = np.array([[1, -1],
4               [1,  1]])
5 b = np.array([1, 3])
6
7 x = np.linalg.inv(A) @ b
8 print(x)  # [2.  1.] ← x=2, y=1
```

💡 実は連立方程式用の関数が存在

$Ax = b$ の解は `np.linalg.solve(A, b)`

まとめ

概念	数式	NumPy
ベクトル	$\mathbf{a}$	<code>np.array([1, 2])</code>
内積	$\mathbf{a} \cdot \mathbf{b}$	<code>a @ b</code>
行列積	AB	<code>A @ B</code>
転置	A^T	<code>A.T</code>
直交	$\mathbf{a} \cdot \mathbf{b} = 0$	<code>a @ b == 0</code>
逆行列	A^{-1}	<code>np.linalg.inv(A)</code>
連立方程式	$Ax = b$	<code>np.linalg.solve(A, b)</code>
ノルム	$\ \mathbf{a}\ $	<code>np.linalg.norm(a)</code>
単位ベクトル	$\hat{\mathbf{a}} = \mathbf{a} / \ \mathbf{a}\ $	<code>a / np.linalg.norm(a)</code>



(1) 以下の2つのベクトルの内積を求め、直交しているか判定せよ。

$$\mathbf{a} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} -1 \\ 2 \end{pmatrix}$$

(2) 以下の行列の掛け算を手計算で求め、NumPyで確認せよ。

$$\begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

(3) 以下の連立方程式を `np.linalg.solve()` で解け。また、`np.linalg.solve()` を使わずに逆行列でもとき、結果を比較せよ

$$\begin{cases} x + y = 4 \\ x - y = 2 \end{cases}$$

(4) $\mathbf{a} = \begin{pmatrix} 6 \\ 8 \end{pmatrix}$ のノルムと単位ベクトルを求めよ。

▶ 解答例を見る

26

その他

27

ベクトルの大きさ（ノルム）

- ベクトルの大きさを **ノルム**（norm、L2ノルム）と呼ぶ

→ $|\mathbf{a}|$ や $\|\mathbf{a}\|$ と書く

$$\mathbf{a} = \begin{pmatrix} 3 \\ 4 \end{pmatrix} \quad \|\mathbf{a}\| = \sqrt{3^2 + 4^2} = 5$$

```
1 import numpy as np
2
3 a = np.array([3, 4])
4 print(np.linalg.norm(a)) # 5.0
```

Python

💡 他にも色々あるノルム

- L1ノルム**（マンハッタン距離）： $\sum |a_i|$
- L2ノルム**（ユークリッド距離）： $\sqrt{\sum a_i^2}$ ← 最もよく使う
- フロベニウスノルム**：行列版のL2ノルム

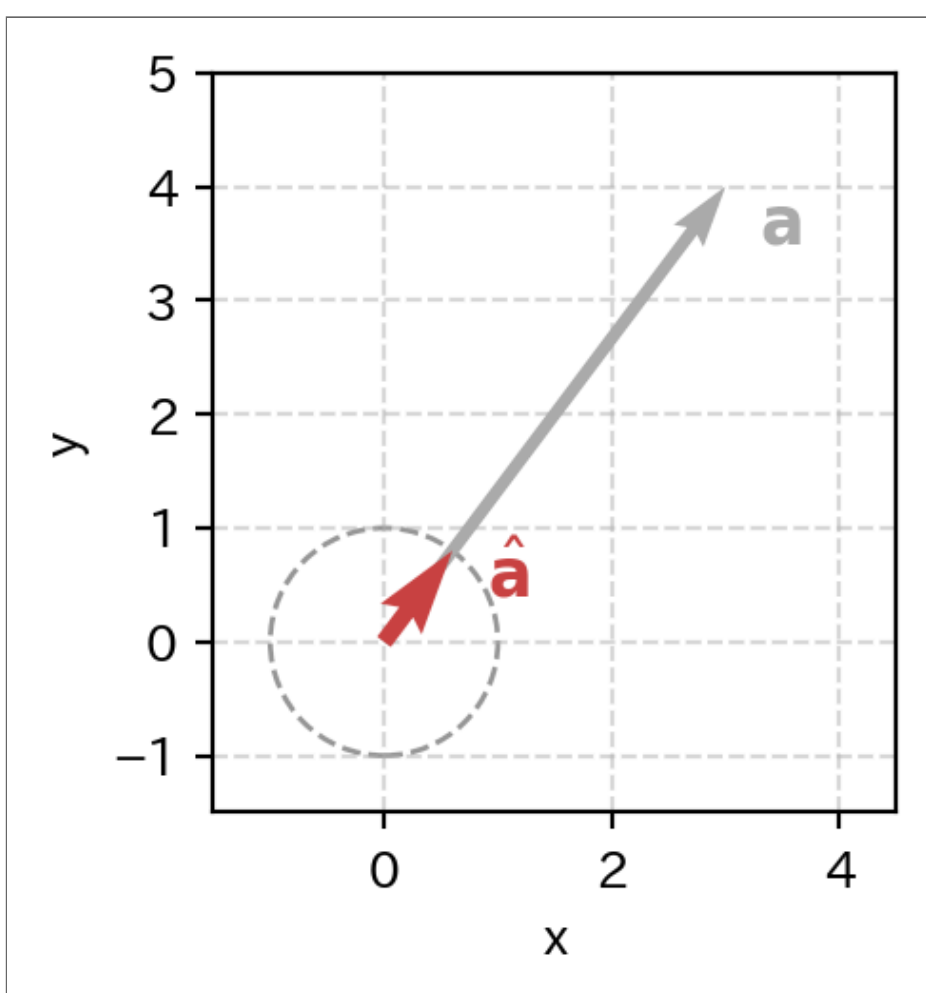
28

単位ベクトル

- ノルムが1のベクトルを **単位ベクトル** と呼ぶ
- 元のベクトルをノルムで割ると単位ベクトルになる
→ **正規化** という

$$\hat{\mathbf{a}} = \frac{\mathbf{a}}{\|\mathbf{a}\|}$$

```
1 import numpy as np
2
3 a = np.array([3, 4])
4 norm = np.sqrt(a[0]**2 + a[1]**2)
5 a_hat = a / norm
6 print(a_hat)           # [0.6 0.8]
7 print(np.linalg.norm(a_hat)) # 1.0
```



29

行列の転置

- 行と列を入れ替える操作を **転置** (A^T) と呼ぶ
→ ベクトルの転置も実は同じ操作
- NumPyでは **.T** で一発

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \Rightarrow A^T = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$$

```
1 import numpy as np
2
3 A = np.array([[1, 2], [3, 4]])
4 print(A.T)
5 # [[1 3]
6 #  [2 4]]
```

30

ベクトルの転置

- 行と列を入れ替える操作を**転置**（**T**ranspose）と呼ぶ
 - ベクトルでは縦を横に倒す操作となる

$$\mathbf{a} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \Rightarrow \mathbf{a}^T = (1 \ 2)$$

- 転置を使うと、内積は $\mathbf{a}^T \mathbf{b}$ と書ける
 - これなら内積の演算を行列と同じ計算方法で扱える
 - ▶ **行** と **列** の要素同士の積の和

$$\mathbf{a}^T \mathbf{b} = (1 \ 2) \begin{pmatrix} 3 \\ 4 \end{pmatrix} = 1 \times 3 + 2 \times 4 = 11$$

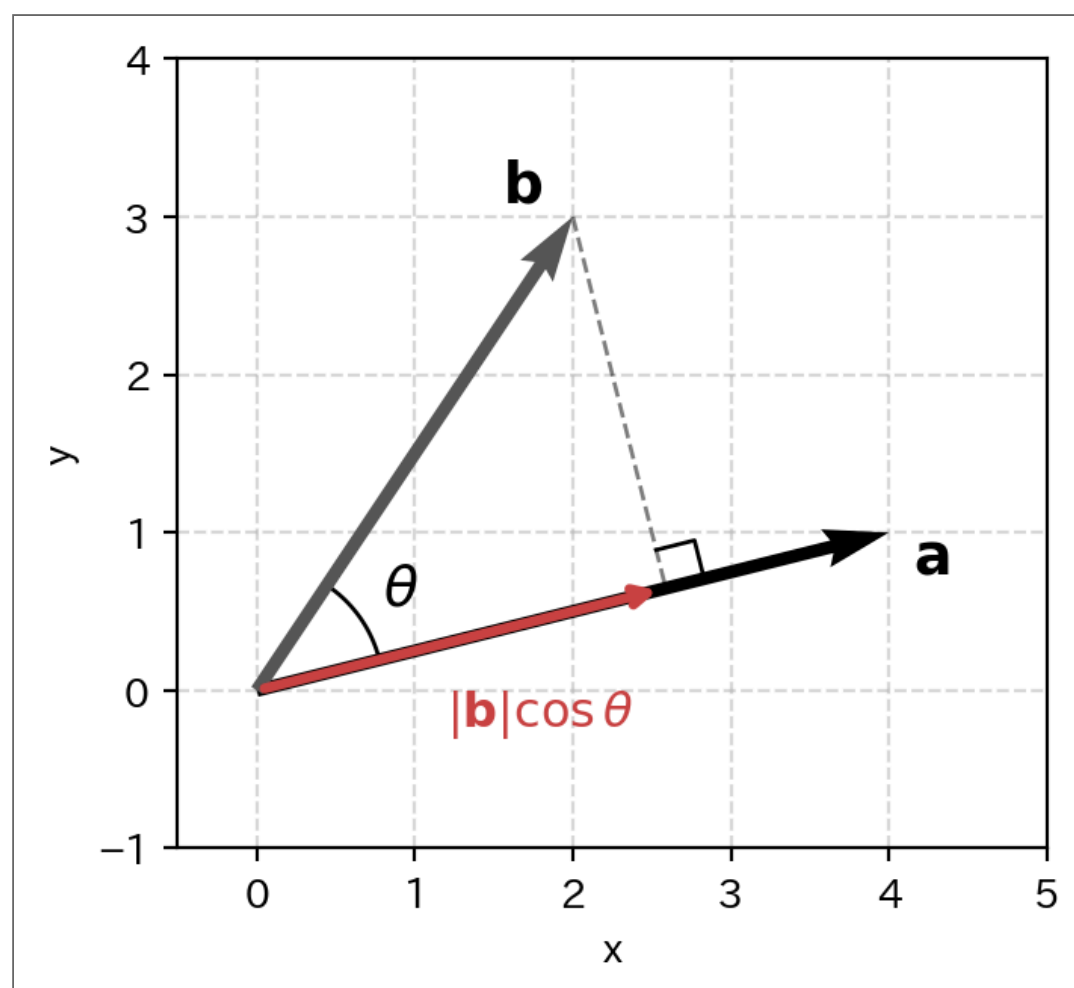
31

図でみる内積

- 教科書での内積の定義

$$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| |\mathbf{b}| \cos \theta$$

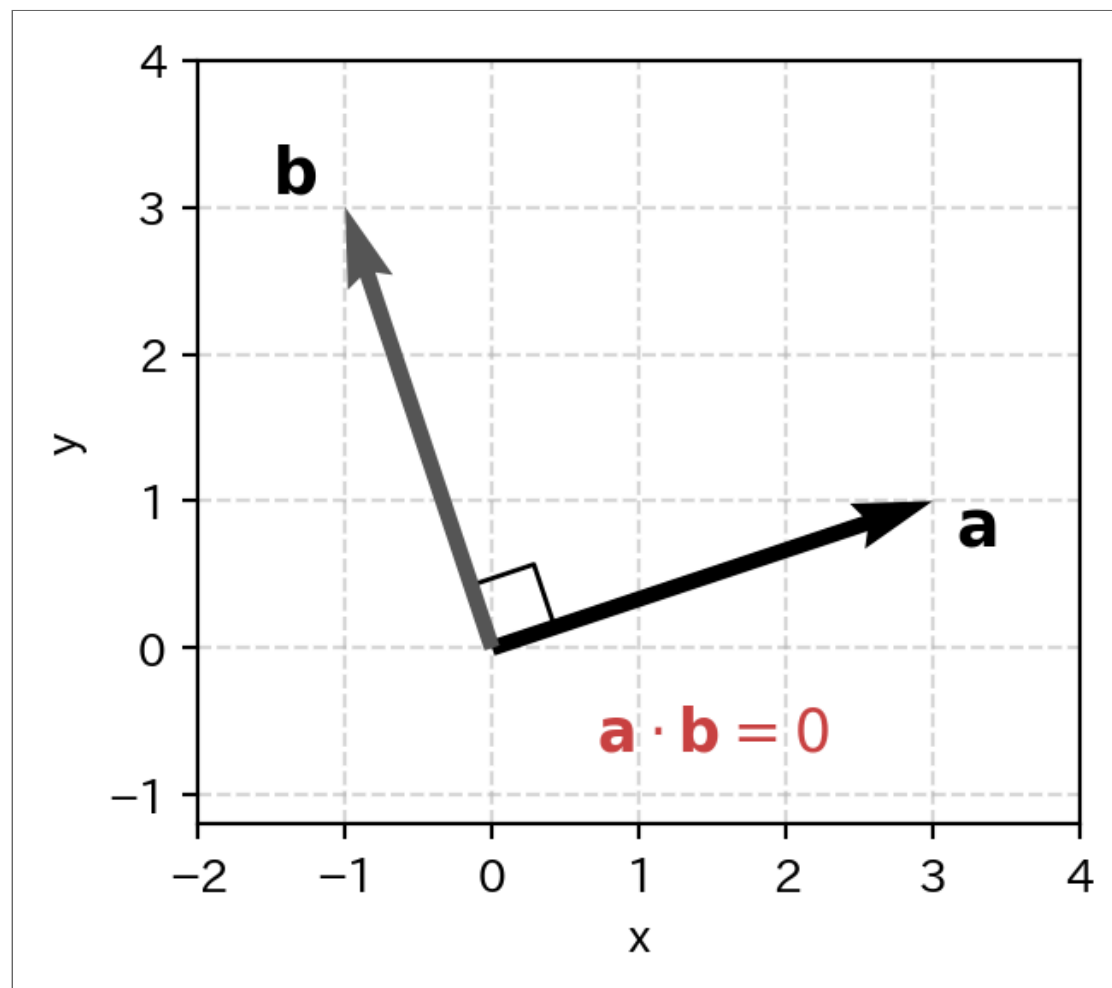
- $|\mathbf{b}| \cos \theta$: $\mathbf{b}$ を $\mathbf{a}$ の方向に**射影**した長さ
- 内積 = 「 $\mathbf{b}$ の $\mathbf{a}$ 方向に対する成分」× 「 $\mathbf{a}$ の長さ $|\mathbf{a}|$ 」



32

内積と直交

- 2つのベクトルが**直交**（90°）しているとき内積は **0**
 - $\cos 90^\circ = 0$
 - つまり、射影の長さがゼロ → 「**a** 方向の成分がない」



① 覚えること

内積 = 0 $\iff$ 直交（垂直）

33

ベクトルの掛け算 3（外積）

- 外積**（cross product）は3次元ベクトル同士から **新しいベクトル** を作る
 - 内積：ベクトル → **スカラー**
 - 外積：ベクトル → **ベクトル**
- 結果は元の2つのベクトルに **直交** する方向を向く
- NumPyでは **np.cross(a, b)**

$$\mathbf{a} \times \mathbf{b} = \begin{pmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{pmatrix}$$

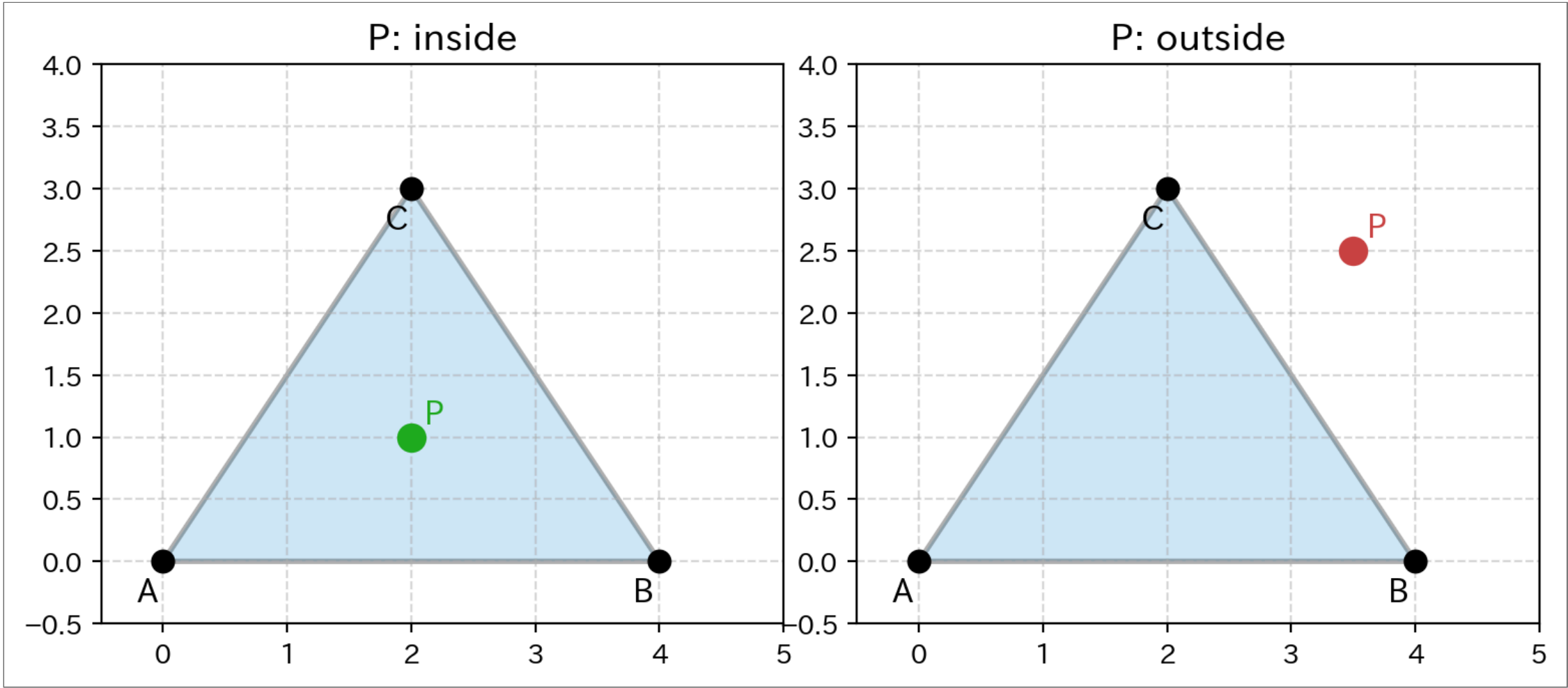
- $|\mathbf{a} \times \mathbf{b}| = |\mathbf{a}| |\mathbf{b}| \sin \theta$ （2つのベクトルが張る **平行四辺形の面積**）

```
1 import numpy as np
2
3 a = np.array([1, 0, 0])
4 b = np.array([0, 1, 0])
5
6 print(np.cross(a, b)) # [0 0 1] ← a, b 両方に垂直
```

Python

34

応用例：多角形に対する内外判定



- 三角形ABCと点Pがある
- 各辺について「辺のベクトル」と「辺の始点→Pのベクトル」の外積を計算
- 全ての外積が同符号 なら内側、そうでなければ外側

35

内外判定の実装例

$$c_1 = (\overrightarrow{AB}) \times (\overrightarrow{AP}), \quad c_2 = (\overrightarrow{BC}) \times (\overrightarrow{BP}), \quad c_3 = (\overrightarrow{CA}) \times (\overrightarrow{CP})$$

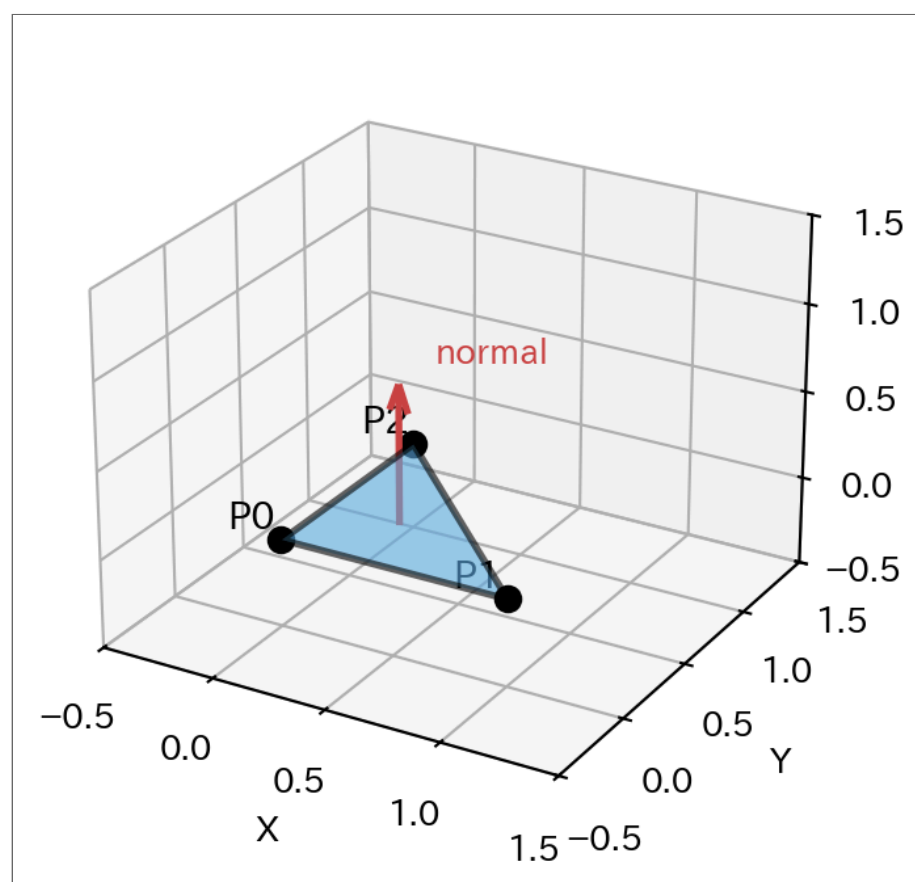
 c_1, c_2, c_3 が全て同符号 → 内側

```
1 import numpy as np
2
3 def cross_2d(v1, v2):
4     """2Dベクトルの外積（z成分のみ）"""
5     return v1[0] * v2[1] - v1[1] * v2[0]
6
7 def is_inside_triangle(A, B, C, P):
8     """点Pが三角形ABCの内側にあるか判定"""
9     # 各辺と点Pへのベクトルの外積
10    c1 = cross_2d(B - A, P - A) # 辺AB
11    c2 = cross_2d(C - B, P - B) # 辺BC
12    c3 = cross_2d(A - C, P - C) # 辺CA
13
14    # 全て同符号なら内側
15    return (c1 >= 0 and c2 >= 0 and c3 >= 0) or (c1 <= 0 and c2 <= 0 and c3 <= 0)
16
17 # テスト
18 A = np.array([0, 0])
19 B = np.array([4, 0])
20 C = np.array([2, 3])
21
22 P_in = np.array([2, 1])
23 P_out = np.array([3.5, 2.5])
24
25 print(is_inside_triangle(A, B, C, P_in)) # True
26 print(is_inside_triangle(A, B, C, P_out)) # False
```

36

その他外積の応用例

- ポリゴンの表裏判定（バックフェースカリング）
 - 三角形の頂点 $P_0 \rightarrow P_1 \rightarrow P_2$ から2辺を作り外積 → **法線ベクトル**
 - 法線がカメラを向いていれば**表**、逆なら**裏**（描画しない）



```
1 import numpy as np
2
3 P0, P1, P2 = np.array([0,0,0]), np.array([1,0,0]), np.array([0,1,0])
4 normal = np.cross(P1 - P0, P2 - P0)
5 print(normal) # [0 0 1] ← +Z方向 = 手前向き
```

Python

37

外積を行列積に変換する

- 外積は**特殊な演算** → 行列のパイプラインに乘せにくい
- **歪対称行列**を使うと、外積が**普通の行列積**になる
 - 3DCG・ロボティクス：姿勢制御の **Lie代数** ($SO(3) / se(3)$)
 - 物理シミュレーション：回転行列の微分 $dR/dt = \Omega R$

$$\mathbf{a} \times \mathbf{b} \longrightarrow [\mathbf{a}]_{\times} \cdot \mathbf{b}$$

💡 何が嬉しいか

- 行列積なら **@** で統一的に扱える
- 複数の変換を **行列の連鎖** として一括計算できる

38

歪対称行列 (skew-symmetric matrix)

- ベクトル $\mathbf{a} = (a_1, a_2, a_3)$ に対応する歪対称行列：

$$[\mathbf{a}]_{\times} = \begin{pmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{pmatrix}$$

- 性質： $A^T = -A$ (転置すると符号反転)、対角成分は常に 0

```
1 import numpy as np
2
3 def skew(a):
4     """ベクトルから歪対称行列を作成"""
5     return np.array([[0, -a[2], a[1]],
6                      [a[2], 0, -a[0]],
7                      [-a[1], a[0], 0]])
8
9 a = np.array([1, 2, 3])
10 b = np.array([4, 5, 6])
11
12 print(np.cross(a, b))    # [-3  6 -3] 外積
13 print(skew(a) @ b)      # [-3.  6. -3.] 行列積で同じ結果
```

Python